IEPS - Assignment 2 report

Tilen Lampret Pika Križnar Grega Radež

Faculty of Computer and Information Science, Ljubljana

Abstract

This report presents the PA2 extraction and retrieval pipeline built on the PA1 GitHub crawl. We extract only the rendered README subtree from each repository page, normalise it into plain text, and segment it with a heading-aware strategy that preserves section context while staying within model token limits. We embed the final 37,914 segments with `sentence-transformers/all-MiniLM-L6-v2` into `pgvector` (`vector(384)`, HNSW cosine index) and query them through a Python demo with optional cross-encoder reranking. We additionally evaluate `LaBSE` (768-dimensional) as an alternative embedding model and compare cosine, L2, inner product, and L1 similarity metrics. Of the 5,000 crawled HTML pages, 4,031 contain a README article (the remainder are commit pages, empty repositories, or other non-landing pages); of those, 4,009 yield at least one segment (the remaining 22 have an empty README). The system performs well on usage-oriented repository questions and exposes expected failure modes on time-sensitive, regulatory, or cross-benchmark comparative queries outside README scope.

Keywords: information extraction – vector retrieval – text segmentation – sentence embeddings – cross-encoder reranking

1 Assignment intent and how we addressed it

The assignment specifies a three-stage pipeline structured around content extraction, vector storage, and semantic retrieval. Beyond this decomposition, the brief lists eight reporting requirements that the PDF must address: filtering criteria, the chunking strategy with its trade-offs, the embedding choice, the similarity metric, three good and three weak query examples, the reranking model, an honest failure case after reranking, and a discussion of methods explored but dropped. We address each of these requirements in dedicated sections below.

The pipeline targets the `github.com` crawl produced by PA1. The crawl is restricted to repository landing pages, so we do not need a generic content extractor: every page exposes its README in a recognisable subtree, which we isolate with a small priority-ordered set of CSS selectors

before normalising the recovered text. Segmented chunks are persisted to a new `crawldb.page_segment` table. Each chunk is then embedded with `sentence-transformers/all-MiniLM-L6-v2`, written to a `vector(384)` column and `sentence-transformers/LaBSE`, written to `vector(768)` column, and indexed by `pgvector` for cosine similarity search. The retrieval and reranking demo, exposed as `demo.py`, takes a natural-language query and returns the top-k segments together with their source URLs.

The implementation is split across four Python scripts in `pa2/implementation-extraction/`: `extract_readme.py` populates `cleaned_content`; `segment_cleaned_content.py` writes rows to `page_segment` for a chosen segmentation strategy; `embed_page_segments.py` encodes each row's `segment_text+embedding_text` and stores the resulting vectors; and `demo.py` runs the retrieval and optional cross-encoder reranking.

2 Website filtering and README extraction

GitHub repository pages contain large amounts of UI elements (navigation, file browser, metadata, scripts) around the rendered README. We therefore extract only the README article subtree, resolved in priority order as `article.markdown-body[itemprop="text"]`, `article.markdown-body.entry-content`, then `article.markdown-body`. Content outside that subtree is ignored. Of the 5,000 crawled HTML pages, 969 do not expose a README article at all (commit pages, empty repositories, or other non-landing pages); for these we store `cleaned_content = NULL` rather than scraping unrelated DOM text. A further 22 pages contain a README subtree that resolves to empty text after normalisation and are likewise left without segments, yielding 4,009 pages with at least one segment out of the 4,031 that contained a README (coverage 99.5%).

Within the selected subtree, we remove scripts/styles/SVG and normalise prose blocks, lists, code blocks, and tables into plain text. We also apply four rules that improved retrieval quality: nested-list protection (prevents duplicated ToC - Table of Content - text), explicit ToC removal, selective link preservation for weak anchor labels (e.g., `paper (https://...)`), and heading markers `[H1]`..`[H6]` for downstream chunking.

The key edge case is syntax-highlighted code blocks. Naive `<pre>` extraction splits one logical line into many token-level lines because GitHub wraps tokens in nested `<span>` elements. We fix this by reading the ancestor attribute `data-snippet-clipboard-copy-content` when available; that value matches what users copy from the page and preserves the intended layout (Figure 1). If absent, we fall back to standard `<pre>` text extraction.

Methods explored but dropped at extraction time were: (i) canonical cleaned-content mapping via `md5` (kept as optional safeguard, disabled for

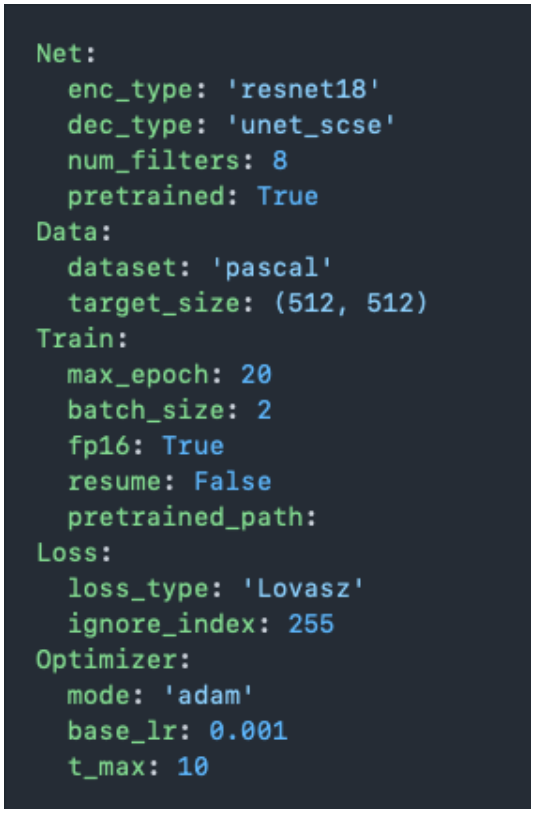

Figure 1: Code block on GitHub README pages. We preserve this rendered layout by reading `data-snippet-clipboard-copy-content` instead of concatenating token-level `<span>` text.

this dataset because PA1 already deduplicated pages), and (ii) whole-body `get_text()` / regex extraction (discarded due to heavy UI-elements pollution).

3 Segmentation strategy

We evaluated three chunking families before selecting our final approach. Strategy A (fixed windows) and Strategy B (paragraph merge) were implemented as baselines, but both mixed unrelated topics on subsection-heavy READMEs. We therefore selected Strategy C (heading-aware, structure-aware), which respects `[H1]..[H6]` boundaries, keeps code/table blocks atomic when possible, and stores metadata (`heading_path`, `segment_type`, `chunk_index`) for ranking and attribution.

The final segmentation pipeline evolved through four versions to resolve specific structural fragmentation issues.

v1 (Semantic Split): The initial heading-aware split lacked token budgeting. This resulted in a median of only 78 tokens and significant micro-fragmentation (14.6% of segments below 20 tokens), with 261 segments overflowing the model's hard cap.

v2 (Tokenizer Budgeting): We introduced strict tokenizer-based budgeting (soft target 200, hard cap 240) using the all-MiniLM-L6-v2 tokenizer to ensure model compatibility. However, v2 suffered from a "buffer-flush" failure mode: if a leading subsection was large (e.g., 220 tokens), v2 would flush the buffer, leaving subsequent small siblings (e.g., 50 and 80 tokens) fragmented even if they could have fit together.

v3 (Multi-pass Repair): We implemented a greedy pack $\rightarrow$ split $\rightarrow$ repair cycle. After a hard split, a "tiny-tail repair" pass re-attaches underfilled fragments to adjacent siblings within the same parent group, provided the merge stays within the hard cap. This reduced micro-segments under 20 tokens from 2.0% to 0.85%.

v4 (Contextual Surfacing): Our production strategy separates `segment_text` (clean display) from `embedding_text` (contextualized model input). v4 adds a surfacing and iterative consolidation loop that promotes "lonely" deep subsections to align with neighbors, recording the hierarchy in a `Nested_scope`: text prefix added at the embedding step. This produced denser, less fragmented chunks while preserving section semantics.

To improve retrieval grounding, each segment is assigned a `segment_type-prose`, `list_bundle`, `code_block`, `table_rows`, or `mixed`. Classification is driven by conservative heuristics: `code_block` is assigned based on fenced markers, indentation patterns, or high punctuation/symbol density (e.g., # [ ] { }). When combining blocks, if the parts share a type, the segment inherits it; otherwise, it is labeled as `mixed`.

The final v4 pipeline executes seven distinct steps: (1) block parsing, (2)

Table 1: Aggregate segmentation statistics across all four strategy versions on the same 4,000-page README corpus snapshot. Token counts use the `all-MiniLM-L6-v2` tokenizer. "<20" and "<40" denote the share of micro-segments below those token counts. ">cap" counts segments exceeding the configured hard cap (420 for v1, 240 for v2 and v3, 256 for the v4 surfacing run).

Strategy	Segments	Median	p95	<20	<40	>cap
v1	42,893	78	311	14.6%	30.2%	261
v2	46,603	149	236	2.0%	8.1%	0
v3	43,303	165	236	0.85%	4.73%	0
v4 (merge-only)	39,998	188	246	0.27%	2.69%	0
v4 (surfacing)	37,914	200	252	0.16%	1.56%	61

	segment_text text	token_estimate integer	char_count integer
96	### Semi-Supervised GAN	19	70
97	### Authors Augustus Odena ### Abstract We extend Generative Adversarial Networks (GANs) to the semi-supervised context by forcing the discriminator network to output class labels. We train a ge…	161	671
98	### Softmax GAN	11	27
99	### Authors Min Lin ### Abstract Softmax GAN is a novel variant of Generative Adversarial Network (GAN). The key idea of Softmax GAN is to replace the classification loss in the original GAN with a …	220	988
100	### StarGAN	28	104
101	### Authors Yunjey Choi, Minje Choi, Munyoung Kim, Jung-Woo Ha, Sunghun Kim, Jaegul Choo ### Abstract Recent studies have shown remarkable success in image-to-image translation for two dom…	207	1038
102	### Run Example	50	177
103	### Super-Resolution GAN	26	109
104	### Authors	57	178
105	### Abstract Despite the breakthroughs in accuracy and speed of single image super-resolution using faster and deeper convolutional neural networks, one central problem remains largely unsolved: h…	228	1068
106	The adversarial loss pushes our solution to the natural image manifold using a discriminator network that is trained to differentiate between the super-resolved images and original photo-realistic ima…	145	696
107	[Paper] [Code] https://camo.githubusercontent.com/b6bddda7b13aaf28adaf7c8a86c7cb7f6a0478267084ee28c8d142b83374dce0/687474703a2f2f6572696b6c696e6465726e6f72656e2e73652f696…	145	212
108	### Run Example	43	137
109	### UNIT	18	57
110	### Authors Ming-Yu Liu, Thomas Breuel, Jan Kautz ### Abstract Unsupervised image-to-image translation aims at learning a joint distribution of images in different domains by using images from th…	211	1088
111	### Run Example	49	148
112	### Wasserstein GAN	13	35
113	### Authors Martin Arjovsky, Soumith Chintala, Léon Bottou ### Abstract We introduce a new algorithm named WGAN, an alternative to traditional GAN training. In this new model, we show that we ca…	134	628
114	### Wasserstein GAN GP	18	60
115	### Authors Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, Aaron Courville ### Abstract Generative Adversarial Networks (GANs) are powerful generative models, but suffer from t…	220	988
116	### Run Example	26	66
117	### Wasserstein GAN DIV	19	55
118	### Authors	30	78
119	### Abstract In many domains of computer vision, generative adversarial networks (GANs) have achieved great success, among which the fam- ily of Wasserstein GANs (WGANs) is considered to be st…	212	891
120	Also, we study the quantitative and visual performance of WGAN-div on standard image synthesis benchmarks, showing the superior performance of WGAN-div compared to the state-of-the-art methods.	46	193
121	[Paper] [Code]	8	14
122	### Run Example	28	68

Figure 2: Residual v2 fragmentation that motivated v3 combine/split repair.

section unit building, (3) parent-aware greedy packing, (4) refine loop with tail repair, (5) final hard split, (6) surfacing and iterative consolidation, and (7) SQL row emission. This ensures that merges reflect contiguity in reading order; we strictly follow an adjacency rule where consolidation never skips a segment with a different `merge_group_parent` to maintain local topical coherence.

Table 1 summarizes the progression across all four versions.

Overall, v4 is our production strategy: fewer micro-segments, higher median chunk density, and better retrieval context than v1 through v3. The main trade-off is higher implementation complexity and tokenizer-cost compared with simpler fixed-window chunking.

4 Embeddings

We embed each `embedding_text` (or `segment_text` for pre-v4 strategies) with two models. The production model is `sentence-transformers/all-MiniLM-L6-v2`, an English-optimised model that produces 384-dimensional vectors and offers a good quality/speed trade-off for documentation-style text. We additionally embed all segments with `sentence-transformers/LaBSE` (Language-agnostic BERT Sentence Embedding), which produces 768-dimensional vectors. Both embeddings are stored in `page_segment` and exposed through `demo.py` via the `-embedding` flag (`minilm` or `labse`).

We use a cosine HNSW index (`vector_cosine_ops`) for each embedding column. HNSW was chosen over IVFFlat because this workload is mostly bulk-build then read/query, where quality and recall are more important than fastest rebuild time.

The embedding script uses batch retries, transactional writes, and basic health diagnostics (norm distribution, near-zero vectors, sparse-vector count) to keep runs reproducible and debuggable.

Alternatives explored but dropped: OpenAI `text-embedding-ada-002` (paid/API-dependent, less reproducible for grading) and multi-model parallel indexing (deferred). The trade-offs between MiniLM and LaBSE are discussed in Section 4.1.

4.1 Embedding model comparison: MiniLM vs. LaBSE

Beyond the production MiniLM model we evaluated `LaBSE` (Language-agnostic BERT Sentence Embedding), a higher-dimensionality model producing 768-dimensional vectors versus MiniLM's 384. The comparison is illustrated on two representative queries run with `-metric cosine -top-k 10 -strategy heading_structure_v4 -rerank`.

On a broad domain query such as *"What are some of the repositories for food image classification?"*, MiniLM surfaced several distinct food-classification repositories, yielding on-topic and diverse results across the corpus. LaBSE, by contrast, concentrated six of its ten top hits on a single repository, demonstrating high recall for one page but poor breadth. The last two reranked hits returned by LaBSE were also less on-topic than those returned by MiniLM.

On a narrow, terminology-rich query such as *"How do I run Segment Anything (SAM) inference or download SAM checkpoints from this repository?"*, LaBSE performed better: it reliably surfaced the official `facebookresearch/segment-anything` repository and concentrated its top hits on segments from that single authoritative source. MiniLM produced a broader, more variant-heavy result set, which is useful when the user wants an overview across the domain but less useful when a precise, focused answer is sought.

The practical recommendation is therefore: use MiniLM for diversified, exploratory queries where breadth across many repositories is desirable, and use LaBSE for narrow, terminology-rich queries where concentrated recall on the single most authoritative source is more valuable than variety.

4.2 Similarity metric comparison

`demo.py` exposes four similarity operators: `cosine`, `l2`, `inner_product`, and `l1` (Manhattan distance). We compared them on the query *"How do I run Segment Anything (SAM) inference or download SAM checkpoints from this repository?"* with `-top-k 3 -strategy heading_structure_v4`:

```
python demo.py --query "How do I run Segment Anything (SAM) \
  inference or download SAM checkpoints from this repository?" \
  --metric <l1|l2|cosine|inner_product> \
  --top-k 3 --strategy heading_structure_v4
```

Cosine, L2, and inner product returned identical top-3 orderings. Cosine is our default because the HNSW index is built with `vector_cosine_ops`, giving it the benefit of approximate nearest-neighbour index acceleration rather than an exact sequential scan. L1 was the only metric that produced a different result: the top hit was a different segment, and the remaining hits were reshuffled relative to the other three metrics. This reshuffling did not yield a clear quality improvement for MiniLM-embedded vectors, so we retain cosine as the production default.

5 Retrieval, queries, and reranking

`demo.py` embeds a query, searches `embeddings`, and returns top-k segments with source URLs. Similarity metrics and embedding models are discussed in Section 4.2 and Section 4.1 respectively. Cosine is our default metric because the ANN index is `vector_cosine_ops`.

GitHub READMEs often contain long citation sections. With embedding-only ranking and small top-k, these can dominate results. We therefore keep a larger candidate pool (`-candidate-k 20`) and rerank before final display. We intentionally keep references in the corpus, because queries about cited papers are legitimate and useful for PA3-style RAG.

5.1 Good queries

Evaluation metrics query. Asking about evaluation metrics yields high-quality results. The first hit explains why mIoU can be unstable, and other reranked hits contain code snippets showing how to compute these metrics on a validation set.

```
python demo.py \
  --query "How do I evaluate mIoU or accuracy on the validation set?" \
  --metric cosine --top-k 10 \
  --strategy heading_structure_v4 --rerank
```

Food image classification repositories. Even though READMEs do not explicitly label themselves as food-classification repositories, the query returns results rich with repository URLs covering that domain. A RAG system built on top of this index can therefore inform the user which repositories are relevant to food image classification. Because references are retained in the database, relevant citation segments are also among the highest-ranked hits, enriching the context with links to domain research.

```
python demo.py \
  --query "What are some of the repositories for food image classification?" \
  --metric cosine --top-k 10 \
  --strategy heading_structure_v4 --rerank
```

SAM inference and checkpoint download. The top reranked hit contains a concrete code fragment showing how to download a checkpoint and run the demo. This single fragment already covers the query fully, and the remaining hits are likewise relevant, providing further detail on downloading and running the Segment Anything model.

```
python demo.py \
  --query "How do I run Segment Anything (SAM) inference \
           or download SAM checkpoints from this repository?" \
  --metric cosine --top-k 10 \
  --strategy heading_structure_v4 --rerank
```

5.2 Weak queries

Best model in 2026. Although the retrieved hits may appear relevant on the surface, they use words such as "current" without any connection to the year 2026. The corpus cannot represent the state of the field as of a specific future or recent date, so results are misleading rather than informative.

```
python demo.py \
  --query "What is the best segmentation model for \
           medical biology images in 2026?" \
  --metric cosine --top-k 10 \
  --strategy heading_structure_v4 --rerank
```

Regulatory approval advice. This query falls squarely in the regulatory-advice domain, which is not present in repository READMEs. The system has no signal to ground an answer, and no reranker can recover information that is absent from the corpus.

```
python demo.py \
  --query "Which segmentation model should my hospital buy \
           for regulatory approval in the EU?" \
  --metric cosine --top-k 10 \
  --strategy heading_structure_v4 --rerank
```

Cross-repository benchmark comparison with time constraint. Asking for an exact mIoU figure from a specific repository compared to another model within a specific recent time window does not yield good results. This illustrates two compounding failure modes: highly specific numerical facts are rarely stated verbatim in READMEs, and freshness constraints ("last month") cannot be satisfied by a static corpus.

```
python demo.py \
  --query "What exact mIoU did this repo achieve on Cityscapes \
           compared to Mask2Former last month?" \
  --metric cosine --top-k 10 \
  --strategy heading_structure_v4 --rerank
```

The reranker is `cross-encoder/ms-marco-MiniLM-L-6-v2`, applied to top-20 embedding candidates as $(query, segment\_text)$ pairs. It improves ordering on weak queries where embedding rank is biased toward bibliography blocks, but still fails when the answer is fundamentally outside README scope. Evidence runs are captured in `pa2/figures/eval_baseline.txt` and `pa2/figures/eval_rerank.txt`.

6 Limitations and methods explored but dropped

The dominant limitation is coverage: we only index README text, so questions requiring issue threads, code internals, or external benchmark synthesis may remain unanswered. Reranking cannot recover facts absent from the corpus. Small/empty READMEs remain intentionally segment-empty.

Methods explored but dropped include whole-body extraction, regex-only HTML parsing, canonical-map dedup as a required step, fixed-window chunking, paragraph-only chunking, OpenAI hosted embeddings, and IVF-Flat as the default index. `LaBSE` was evaluated experimentally rather than dropped; its trade-offs relative to MiniLM are documented in Section 4.1.

7 Conclusion

This report presented an information-extraction and vector-retrieval pipeline built on top of the GitHub image-segmentation crawl from PA1. The extractor isolates the rendered README article on each repository page using a small set of priority-ordered selectors, normalises code blocks via

the `data-snippet-clipboard-copy-content` attribute, and emits heading markers that the segmentation stage relies on. The chosen segmentation strategy, refined through four versions, preserves section semantics and metadata, packs sibling subsections under a soft target, splits oversized groups under a hard cap, and consolidates same-parent neighbours when the embedder budget permits. The resulting 37,914 segments are embedded with both `all-MiniLM-L6-v2` (`vector(384)`) and `LaBSE` (`vector(768)`), each with an HNSW cosine index, and the retrieval demo exposes the corpus through a Python program that supports four similarity metrics, two embedding models, and an optional cross-encoder reranking stage.

Among the four similarity metrics, cosine, L2, and inner product produced equivalent rankings; L1 reshuffled results without a clear quality benefit for MiniLM-embedded vectors, confirming cosine as the appropriate default given the indexed operator. An experimental comparison between MiniLM (384 dimensions) and LaBSE (768 dimensions) revealed a consistent trade-off: MiniLM delivers broader, more diverse retrieval across the corpus, while LaBSE concentrates hits on the single most authoritative source for narrow, terminology-rich queries.

The pipeline grounds task-oriented queries about evaluation metrics, usage instructions, and pretrained-model setup on the right README sections. The failure modes on time-sensitive, regulatory, and cross-benchmark comparative queries are honestly reported as open limitations. The dataset and pipeline are now ready to feed PA3's retrieval-augmented generation layer.